SentinelGraph

Security Audit Report

Findings, prediction metrics, deployment readiness, and lab validation

Prepared by Manus AI

Audit date: 2026-10-05

Scope: isolated synthetic SentinelGraph lab and prototype services

Contents

1 Executive summary 1
2 Scope and methodology 1
2.1 In scope 1
2.2 Out of scope 1
2.3 Evidence basis 2
3 Architecture and trust boundaries 2
4 Graph findings 2
4.1 Baseline metrics 2
4.2 Security interpretation 3
5 Investigation and evidence findings 3
6 Prediction metrics 3
7 What-if simulation findings 4
8 Test and validation evidence 4
9 Kubernetes and Helm deployment readiness 4
10 Live attack scenario status 5
11 Risk register and recommended next steps 5
12 Conclusion 6

1 Executive summary

SentinelGraph is an explainable attack-path intelligence prototype. It combines a NetworkX graph model, FastAPI alert ingestion, a deterministic evidence-grounded investigation loop, a React/Cytoscape dashboard, graph-copy What-if simulation, and transparent attack prediction.

The fresh synthetic baseline contains 22 enumerated paths from the Internet entry point to two crown jewels: the production database and secrets vault. The baseline risk score is **85/100**, derived from 22 paths and an average weighted path likelihood of 0.927. The edge gateway is the dominant choke point: all 22 paths pass through it.

The prototype demonstrates strong explainability and safety boundaries, but the assessment is not a production penetration test. Inventory, CVE, EPSS, and KEV values are simulated fixtures. The Compose lab and detector are designed for local validation only. Docker, Kubernetes, and Helm were unavailable in the execution sandbox, so live container-health and cluster deployment claims could not be made.

Assessment area	Result	Interpretation
Attack-path baseline	22 paths / 2 jewels	High path concentration; edge gateway is a systemic choke point.
Risk score	85 / 100	Path-weighted prototype score, not a production risk rating.
Prediction	3 ranked moves	Top score: backup-service at 0.669.
Investigation evidence	100% coverage	Each verdict claim links to tool-produced evidence IDs.
Test suite	12 passed	Backend graph, alert, investigation, simulation, and prediction tests pass.
Live infrastructure	Blocked	Docker, kubectl, and Helm are not installed in the sandbox.

2 Scope and methodology

2.1 In scope

- Synthetic hosts, users, roles, vulnerabilities, network relationships, entry points, and crown jewels.
- NetworkX path enumeration, choke-point ranking, and risk scoring.
- FastAPI alert queue, SSE streams, investigation tools, and verdict evidence validation.
- React dashboard routes: Attack Graph, Top Risks, Investigation, and What-if.
- Graph-copy simulation actions and the PRD prediction formula.
- Docker Compose and Helm deployment definitions as source artifacts.

2.2 Out of scope

- Any external target, real enterprise inventory, real CVE/EPSS feed, or production credential.
- Live Falco/Wazuh runtime verification; the detector is a portable local log-tailing fallback.
- Kubernetes cluster deployment because no cluster context or Kubernetes CLI is available.

- Automatic remediation. Recommendations remain human-gated.

2.3 Evidence basis

The report uses a fresh `SyntheticLab()` baseline captured from the restarted FastAPI service on 2026-10-05. Dynamic test output was independently rerun during this audit. A prior demo approval can mutate the in-memory graph; therefore baseline metrics are explicitly separated from any mutated live session.

3 Architecture and trust boundaries

The prototype has four logical planes:

1. **Graph plane:** `SyntheticLab` builds the directed NetworkX graph, enumerates simple entry-to-crown paths, and calculates choke-point coverage and path-weighted risk.
2. **Detection plane:** the isolated Compose victim writes SSH-like authentication events. The local detector tails only the shared victim log and posts normalized alerts to FastAPI.
3. **Reasoning plane:** the investigation manager runs `query_logs`, `query_graph`, `lookup_cve`, `map_attack_technique`, and `get_asset_context`. Tool results carry evidence IDs; unsupported claims are filtered.
4. **Presentation and change plane:** React consumes graph, alert, investigation, simulation, and prediction APIs. What-if changes operate on a graph copy. Approval is a separate explicit endpoint.

The attacker script hard-codes the Compose hostname `victim` and port `2222`. The Compose network is marked `internal: true`. These are appropriate prototype guardrails, but they should be reinforced with network policies, service accounts, admission controls, and runtime egress restrictions in Kubernetes.

4 Graph findings

4.1 Baseline metrics

The fresh graph contains 15 nodes and 21 edges, including explicit `HAS_VULN` relationships. It enumerates 22 attack paths to the two crown jewels. The risk formula is:

`min(100, (paths / 24) × average weighted path likelihood × 100)`

The resulting baseline is 85/100 with average weighted path likelihood 0.927.

Node	Paths	Coverage	Betweenness	Finding
Edge gateway	22 / 22	100%	0.071	Primary choke point; every path crosses the gateway.
Application server	18 / 22	81.8%	0.118	High-value pivot into database, vault, and backup service.
App admin role	16 / 22	72.7%	0.085	Privilege-bearing IAM pivot that reaches the application tier.

Web portal	14 / 22	63.6%	0.082	Public-facing service with the highest exploitability fixture.

4.2 Security interpretation

The edge gateway is the best first remediation candidate because it provides maximum path coverage. The application server is the strongest internal pivot candidate by betweenness and connects multiple crown-jewel routes. The app admin role is a privilege concentration risk. The web portal has a high exploitability signal and should be treated as an urgent exposure in the synthetic scenario.

These findings are graph-derived, not exploit proof. A production assessment would validate asset ownership, reachability, authentication controls, segmentation, and compensating controls before assigning a business risk rating.

5 Investigation and evidence findings

The alert webhook accepts a normalized alert and starts a deterministic ReAct-style investigation. A live end-to-end API run produced 13 events and invoked all five tools:

Tool	Purpose	Evidence output
`query_logs`	Correlate failed and successful authentication events	`log-001` through `log-004`
`query_graph`	Map the alert host to graph topology and paths	`graph-vpn-bastion`
`lookup_cve`	Retrieve offline vulnerability fixture context	`cve-CVE-2024-3400`
`map_attack_technique`	Map the alert rule to ATT&CK	`attack-t1110`
`get_asset_context`	Map the container host to the synthetic asset	`asset-victim`

The generated verdict was **confirmed attack** with 89% confidence and 100% evidence coverage. Claims without complete evidence IDs are removed before display. The recommendation is advisory only: block the attacker source at the lab edge and rotate the victim credential, subject to human approval.

6 Prediction metrics

Prediction ranks up to three reachable next moves using the PRD formula:

`score(c) = 0.4 × exploitability(c) + 0.3 × privilege_gain(c) + 0.3 × proximity_to_crown_jewel(c)`

The current source host `victim` maps to the synthetic `vpn-bastion` foothold. Scores below were recomputed independently from the returned components.

Candidate	Score	Exploitability	Privilege gain	Crown proximity	Techniques

Backup service	0.669	0.736	0.250	1.000	T1021, T1190
Web portal	0.543	0.982	0.000	0.500	T1021, T1190
App admin role	0.515	0.350	0.250	1.000	T1078

The ranking is intentionally explainable. It is not an ML prediction and the LLM does not select the order. Exploitability is a normalized fixture-derived signal using severity, EPSS, and KEV bonus; privilege gain compares the candidate privilege level with the foothold; proximity uses inverse directed shortest-path distance to a crown jewel.

7 What-if simulation findings

The simulation engine creates a deep graph copy, applies one of four operations, recomputes paths and risk, and returns removed paths and score deltas. The baseline is not mutated until a separate approval call. A representative live simulation patched the web portal fixture `CVE-2021-44228`:

- Baseline: 22 paths, risk 85.
- Simulation copy: 8 paths, risk delta −55.
- Baseline after preview: unchanged.
- Approval: explicit and demo-only; no external remediation is executed.

Supported actions are vulnerability patching, port closure, IAM role revocation, and edge removal. The dedicated unit suite now covers all four operations, including close-port and revoke-role isolation behavior.

8 Test and validation evidence

Validation	Result	Notes
Backend unit suite	12 passed	One non-blocking Starlette/AnyIO deprecation warning.
Python compilation	Passed	Backend and lab modules compile successfully.
Frontend production build	Passed	Vite build completed; bundle-size warning only.
Dashboard routes	Passed	All four routes returned HTTP 200.
API integration	Passed	Alert, investigation, prediction, and simulation flow verified.
Docker Compose runtime	Blocked	Docker CLI unavailable in sandbox.
Kubernetes deployment	Blocked	kubectl, Helm, and cluster context unavailable.

9 Kubernetes and Helm deployment readiness

A Helm chart has been added under `deploy/helm/sentinelgraph`. It defines separate backend and frontend Deployments and Services, restricted security contexts, readiness/liveness probes, configurable images, and optional Ingress. The frontend image uses Nginx to serve the built React bundle and proxy `/api/` and `/alerts` to the backend Service.

The chart is deployment-ready as a source artifact, but it was not installed because this sandbox has no `helm`, `kubectl`, or Kubernetes context. Before deployment, build and push both images to a registry, set `backend.image.repository`, `frontend.image.repository`, and tags in a values file, then run:

```
helm lint deploy/helm/sentinelgraph
helm upgrade --install sentinelgraph deploy/helm/sentinelgraph \
  --namespace sentinelgraph --create-namespace \
  --set backend.image.repository=REGISTRY/sentinelgraph-backend \
  --set frontend.image.repository=REGISTRY/sentinelgraph-frontend
kubectl -n sentinelgraph rollout status deployment/sentinelgraph-backend
kubectl -n sentinelgraph rollout status deployment/sentinelgraph-frontend
```

The chart does not include the attacker container by design. The attacker remains a controlled lab-only job and should be run only in a dedicated test namespace with explicit network policy.

10 Live attack scenario status

The requested live attack scenario against the Docker Compose lab could not be executed in this environment because Docker and Docker Compose are not installed. Static Compose validation passed previously: the lab defines `platform`, `victim`, `detector`, and `attacker`, and the network is internal. The safe execution procedure is:

```
docker compose up -d platform victim detector
./demo.sh
docker compose ps
docker compose logs --no-color detector
curl http://127.0.0.1:8000/api/alerts
docker compose down -v
```

Expected evidence is a victim authentication event, a detector POST to `/alerts`, a queued alert, and a streamed investigation. Do not substitute external targets or arbitrary hostnames.

11 Risk register and recommended next steps

ID	Risk	Recommendation	Priority
R-01	Edge gateway is on 100% of paths	Validate segmentation and add a choke-point remediation playbook.	High
R-02	Web portal has high exploitability fixture	Patch, isolate, and validate public exposure in a real inventory.	High
R-03	Admin role concentrates privilege	Review role grants and require least privilege / JIT access.	High
R-04	Live infrastructure was not exercised	Run Compose and Helm checks on Docker/Kubernetes-enabled CI.	Medium
R-05	In-memory state is not durable	Persist alerts, investigations, approvals, and graph versions before production use.	Medium

12 Conclusion

SentinelGraph provides a credible prototype foundation for explainable attack-path analysis: graph results are inspectable, investigation claims are evidence-linked, What-if changes are copy-based and human-approved, and predictions expose their component scores. The most important next assurance step is not a code feature; it is controlled runtime validation on a Docker-enabled host and a Kubernetes cluster, followed by replacing simulated threat fixtures with authoritative, time-bounded data sources.